

Reviewer Pack — Free Entropy Gap in Read-Twice Branching Programs for IP₂

Entry 0b9d1a98a4f1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 11:14:32 UTC

Free Entropy Gap in Read-Twice Branching Programs for IP₂

Entry ID: 0b9d1a98a4f1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 11:14:32 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

For a read-twice branching program P computing IP_2 (inner product mod 2), let μ_P be the empirical spectral distribution of its transition matrix. The free entropy $\phi(\mu_P) = -\int \log|z| d\mu_P(z)$ satisfies $\phi(\mu_P) = \Theta(\log n)$ for trivial P (e.g., constant functions), but $\phi(\mu_P) \geq \Omega(n)$ for nontrivial P . The gap is witnessed by the spectral radius $\lambda_P = \max\{|z| : z \in \text{spec}(\mu_P)\}$ with $\lambda_P \geq 1 + \Omega(1/n)$ for nontrivial P .

Rationale (proposer's reasoning):

Free entropy quantifies the 'spread' of spectral measures, capturing structural complexity in non-commutative probability spaces. Read-twice BPs for IP_2 exhibit distinct spectral signatures: trivial programs concentrate measure near 1, while nontrivial ones spread it across eigenvalues, creating a detectable free entropy gap.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ad15fddef97e9f08

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(n):
                if j != i:
                    factor = A[j][i] / A[i][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def spectral_radius(matrix):
        n = len(matrix)
        eigenvalues = [1.0] * n
        for _ in range(100): # Power iteration method
            v = [random.random() for _ in range(n)]
            v /= sum(v)
            Av = matrix_multiply(matrix, v)
            lambda_new = max(abs(x) for x in Av)
            if abs(lambda_new - eigenvalues[-1]) < 1e-6:
                break
            eigenvalues.append(lambda_new)
        return eigenvalues

    def free_entropy(eigenvalues):
```

```

        return -sum(math.log(abs(z)) for z in eigenvalues)

n = random.randint(5, 40)
seed += n # Ensure different seeds for different sizes
random.seed(seed)

# Generate a read-twice branching program for IP_2
transition_matrix = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
for i in range(n):
    transition_matrix[i][i] = 1

# Compute the spectral radius and free entropy
eigenvalues = spectral_radius(transition_matrix)
lambda_P = max(abs(z) for z in eigenvalues)
phi_mu_P = free_entropy(eigenvalues)

# Determine if the program is trivial or nontrivial
is_trivial = all(x == 1 for x in transition_matrix[0])
conjecture_holds = (is_trivial and phi_mu_P == math.log(n)) or (not is_trivial and phi_mu_P >= 0.8)
counterexample = "mapping_undefined" if not conjecture_holds else ""

return {
    "metric_name": "Free Entropy Gap",
    "metric_value": phi_mu_P,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")

    phi_mu_P_values = [r["metric_value"] for r in results]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(phi_mu_P_values)/len(phi_mu_P_values):.6f} std=0 support={support_fraction:.6f}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(phi_mu_P_values)/len(phi_mu_P_values):.6f} std={math.sqrt(sum((x - sum(phi_mu_P_values)/len(phi_mu_P_values))**2 for x in phi_mu_P_values))/len(phi_mu_P_values):.6f} support={support_fraction:.6f}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc3e2548.py", line 89, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc3e2548.py", line 66, in run_trial
    eigenvalues = spectral_radius(transition_matrix)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dc3e2548.py", line 45, in spectral_radi
    v /= sum(v)
TypeError: unsupported operand type(s) for /=: 'list' and 'float'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed during spectral radius computation, preventing data collection. No results available to assess conjecture. | next: Implement numerical stability fixes for spectral radius calculation and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111529
2	propose	ollama_remote	qwen3:8b	0	0	38072
3	propose	ollama_remote	qwen3:8b	0	0	37933
4	preregistration	ollama_remote	qwen3:8b	0	0	24149
5	novelty	ollama_remote	qwen3:8b	0	0	20834
6	novelty	ollama_remote	qwen3:8b	0	0	12593
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18682
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11465
9	judge	ollama_remote	qwen3:8b	0	0	15078

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 290336 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0b9d1a98a4f1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/0b9d1a98a4f1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0b9d1a98a4f1.tar.gz` (if generated)